

GL1F Formal and Reproducibility Supplement

Canonical bytes, bounded traversal, exact cross-runtime inference, fixed-point error bounds, code-as-data reconstruction, and deployment evidence.

Mikhail Fedorov
Version dated 5 September 2026 · not peer reviewed

Evidence summary

Cross-engine suite	18/18 test methods; 25 distinct three-engine training profiles plus one standalone arithmetic witness
Formal boundary witnesses	8/8 executable properties
Local EVM integration	v1/v2 boundary parity plus 72/72 scalar scaling comparisons
Provider-attested deployment summary	12/12 hashes and nine named relations; 108/108 local/provider-returned read-call outputs matched
Reported reconstructed model data	31,185,324 bytes across up to 397 exact GL1C chunks per model
Canonical Solidity build	solc 0.8.20, vialR, optimizer 200, Istanbul target

Controlling distinction. A proof of execution equivalence says which integer result follows from fixed bytes and inputs. It does not prove the scientific truth, predictive utility, safety, authorship, or causal meaning of the model.

Evidence boundary. Raw provider responses and reconstructed deployed model/runtime bytes are not included. The live-state values are therefore provider-attested historical summaries; the included source supports a fresh RPC-assisted observation and local reproduction of the remaining evidence.

Contents

1	Purpose and assurance model	3
2	Evidence map	3
3	Formal model and correctness results	3
3.1	Canonical representation	3
3.2	Chunked EVM representation	4
3.3	Traversal and termination	5
3.4	Cross-runtime equivalence	6
3.5	Fixed-point approximation	7
3.6	Limits of what the current registry proves	8
3.7	Proof obligations for a publication-grade deployment	8
4	Executable evidence	9
4.1	Three-engine training parity	9
4.2	Reference performance observation	9
4.3	Isolated EVM integration	9
4.4	Local-EVM scalar scaling	9
5	Pinned deployment witness	10
6	Open-protocol and registry boundary	10
7	Scientific limits and falsifiability	10
8	Reproduction commands	11
9	Conclusion	11

1 Purpose and assurance model

GL1F makes a deliberately narrow model family unusually easy to inspect. A complete fixed-depth tree is serialized as fixed-width integers. The EVM runtime performs no floating-point arithmetic and no data-dependent recursion: it reads a feature index and threshold, applies one strict comparison, updates a heap index, reads one leaf, and accumulates an integer. The same core is consumed by browser, native, and contract evaluators.

The supplement establishes five different claims, each with its own evidence:

1. **Representation:** exact byte geometry follows from the complete tree layout.
2. **Traversal:** a depth- d tree reaches exactly one valid leaf after exactly d decisions.
3. **Reconstruction:** a well-formed GL1C manifest returns the exact serialized primitives, including chunk-boundary reads.
4. **Equivalence:** identical externally verified canonical bytes and packed int32 input produce the same integer scalar or vector under conforming local and EVM evaluator specifications. Source inspection and finite tests separately support the tested contract path.
5. **Approximation:** abstract exact-real fixed-point error is bounded when saturation and path changes are excluded; the production binary64 correction and a split-margin condition make that boundary explicit.

Training parity is not promoted to a universal theorem. The trainers use floating-point split scores, metrics, exponentials/logarithms, and seeded random choices. Their equality is therefore supported by an explicit publication profile and executable cross-engine tests.

2 Evidence map

Class	Meaning	Interpretation
P	Proved	Derived from definitions and stated preconditions.
T	Tested	Reproduced by an executable test or benchmark on the recorded environment.
C	Chain-observed	Read at a pinned block; state and RPC evidence are explicitly scoped.
S	Source fact	Directly implemented or declared in the tested repository.
L	Literature	Supported by primary specifications or peer-reviewed literature.
U	Unsupported	Overbroad, contradicted, or not demonstrated; not presented as established.

Scoped system statement

GL1F is an integrated, proof-friendly, fixed-depth integer GBDT format and tool chain spanning browser, Python, and C++ training; exact local inference; fixed EVM runtime-code publication at a pinned state; and direct scalar or vector contract execution. Under the publication profile, the tested trainers emitted byte-identical cores; for canonical bytes and identical packed input, exact equality between conforming evaluator specifications is proved. Source, test, and chain evidence separately support the tested implementation.

3 Formal model and correctness results

This section separates three objects that are easy to conflate: (i) the mathematical forest, (ii) its GL1F byte representation, and (iii) a registry entry that points to EVM code chunks. The equivalence result for the first two is exact under the representation, input, depth, and arithmetic premises below. Its application to a registry entry is also conditional on a well-formed storage manifest, because the deployed registry does not itself recompute the model hash or validate every serialization invariant.

3.1 Canonical representation

Let $\mathbb{I}_{32} = [-2^{31}, 2^{31} - 1] \cap \mathbb{Z}$. A complete binary tree of depth d contains

$$L(d) = 2^d \text{ leaves and } I(d) = 2^d - 1 \text{ internal nodes.} \quad (1)$$

An internal node occupies eight bytes: a little-endian unsigned 16-bit feature index, a little-endian signed 32-bit threshold, and two reserved bytes. A leaf occupies four bytes containing a little-endian signed 32-bit value. Consequently, one serialized tree occupies

$$b(d) = 8I(d) + 4L(d) = 12 \cdot 2^d - 8 \quad (2)$$

bytes.

Definition 3.1 (Canonical GL1F v1 model). *A canonical publication-profile scalar model is a byte string B consisting of a 24-byte header and T trees, where $F \geq 1$, $1 \leq d \leq 12$, and $0 \leq T \leq 2^{32} - 1$. A zero-tree v1 object is the well-defined base-only case. Its header contains magic GL1F, version one, feature count F , depth d , tree count T , base value $a_0 \in \mathbb{I}_{32}$, and scale $s \in [1, 2^{32} - 1] \cap \mathbb{Z}$. Reserved bytes are zero, every node feature is in $\{0, \dots, F - 1\}$, and*

$$|B| = 24 + Tb(d). \quad (3)$$

For a deployed-registry EVM application we additionally require equality between serialized and registered F, d, T, a_0 , and s , and $T \leq 2^{16} - 1$. The latter is a deployed-interface restriction: the v1 wire field for T is unsigned 32-bit, whereas the deployed registry stores total tree count as unsigned 16-bit.

Definition 3.2 (Canonical GL1F v2 model). *A canonical publication-profile vector model has $F \geq 1$, $1 \leq d \leq 12$, $K \geq 2$ outputs, $R \geq 1$ trees per output, base vector $(a_{0,0}, \dots, a_{0,K-1}) \in \mathbb{I}_{32}^K$, and output-major tree order. Its exact length is*

$$|B| = 24 + 4K + KRb(d). \quad (4)$$

The header has magic GL1F, version two, positive F and s , and zero reserved fields. Every node feature is smaller than F . Registered feature count and scale equal the serialized values. Publication through the deployed registry additionally requires $KR \leq 2^{16} - 1$, because that interface stores the total tree count in a 16-bit field; this is narrower than the core format's separate 16-bit K and 32-bit R fields.

The format does not contain a task discriminator. Identical v1 bytes can be described externally as regression or binary-logit inference, and identical v2 bytes can be described as multiclass or multilabel inference. Here keccak256 denotes Ethereum's legacy Keccak-256 function, not the standardized FIPS SHA3-256 variant. Under the usual collision- and second-preimage-resistance assumptions, keccak256(B) binds a practical identifier to the exact serialized bytes B . It is not a semantic normal form: distinct canonical cores can denote the same extensional integer function. The digest also does not bind a task label, feature semantics, preprocessing contract, dataset, or intended use. Those claims require a separately authenticated semantic manifest.

The explicit depth premise matches the current first-party interface. It is substantially narrower than the 16-bit wire field and prevents an apparently valid header from requesting an impractically large complete tree or an evaluator-specific shift result.

3.2 Chunked EVM representation

Let c be the nominal payload of every nonfinal chunk and therefore the logical byte-offset stride; the final payload may be shorter. Let $N = \lceil |B|/c \rceil$. The canonical publisher deploys each payload through `ModelStore.write`, creating runtime code

$$C_j = \text{GL1C} \parallel B[jc : \min\{(j+1)c, |B|\}].$$

It then creates a table whose data are N 32-byte words; the low 20 bytes of word j are the address of C_j . The table is itself prefixed by GL1C.

Definition 3.3 (Well-formed storage manifest). *For an EVM state σ and registry record r in that state, a manifest (P, c, N, ℓ) is well formed for B at (σ, r) when:*

1. $P \neq 0$, $4 \leq c \leq 24572$, $1 \leq N \leq 767$, $\ell = |B|$, and $N = \lceil \ell/c \rceil$;
2. table address P has runtime code in σ of exact length $4 + 32N$, beginning with GL1C;
3. its j th word has zero high 12 bytes and resolves through its low 20 bytes to nonzero address C_j ;
4. every nonfinal C_j in σ has exact code $\text{GL1C} \parallel B[jc : (j+1)c]$, and the final chunk has the exact code $\text{GL1C} \parallel B[(N-1)c : \ell]$; and
5. the corresponding fields of r equal P, c, N, ℓ .

The lower bound $c \geq 4$ is semantic, not cosmetic. Runtime primitive fields are at most four bytes, while the implemented boundary reader consults at most two chunks. A field could span more than two logical chunks if $c < 4$.

Lemma 3.4 (Pointer capacity). *A pointer table created by the deployed storage contract contains at most*

$$\left\lfloor \frac{24572}{32} \right\rfloor = 767$$

chunk addresses.

Proof. `ModelStore.write` accepts at most 24572 data bytes, and each table slot occupies exactly 32 data bytes. The four-byte GL1C prefix is added by the store and is not part of the payload limit. $\square$

For a manifest satisfying Definition 3.3, define the specified boundary reader $\text{Read}_{\sigma, M}(o, n)$ to compute $j = \lfloor o/c \rfloor$ and $r_o = o \bmod c$, copy from code offset $4 + r_o$ in chunk j , and, only when $r_o + n > c$, concatenate the remaining bytes from code offset four in chunk $j+1$.

Lemma 3.5 (Exact byte reconstruction). *Given Definition 3.3, every runtime primitive read of $n \in \{1, 2, 4\}$ bytes at a model offset $o \in \mathbb{Z}_{\geq 0}$ satisfying $o + n \leq |B|$ returns exactly $B[o : o + n]$.*

Proof. By Euclidean division, write $o = jc + r$ with $j = \lfloor o/c \rfloor$ and $0 \leq r < c$. If $r + n \leq c$, the runtime resolves slot j and copies from code offset $4 + r$, which is exactly $B[o : o + n]$. Otherwise it copies $c - r$ bytes from chunk j and $n - (c - r)$ bytes from the start of chunk $j + 1$. Since $n \leq 4 \leq c$, no third chunk is required. The exact chunk-length and ordering clauses in Definition 3.3 make the concatenation equal to $B[o : o + n]$. $\square$

Remark 3.6. The GL1C check detects zero and many accidental pointers, but it is not content authentication. Arbitrary EVM code beginning with the same four bytes passes that test. Exact reconstruction is therefore a property of the manifest conditions, not of the magic prefix alone.

3.3 Traversal and termination

Let i_j be the heap index after j decisions, with $i_0 = 0$. For packed feature x_q and threshold θ_q , a node updates

$$i_{j+1} = \begin{cases} 2i_j + 2, & x_q > \theta_q, \\ 2i_j + 1, & x_q \leq \theta_q. \end{cases} \quad (5)$$

Lemma 3.7 (Traversal interval). *After j decisions,*

$$2^j - 1 \leq i_j \leq 2^{j+1} - 2. \quad (6)$$

Proof. For $j = 0$, both bounds equal zero. Assume the claim at j . The smallest possible child is $2(2^j - 1) + 1 = 2^{j+1} - 1$, and the largest is $2(2^{j+1} - 2) + 2 = 2^{j+2} - 2$. These are precisely the bounds for $j + 1$. $\square$

Corollary 3.8 (Leaf-index safety). *After exactly d decisions,*

$$0 \leq i_d - I(d) \leq L(d) - 1.$$

Thus a traversal of a well-formed tree selects exactly one serialized leaf and cannot address outside that tree's leaf array.

Proof. Substitute $j = d$ in Lemma 3.7 and subtract $I(d) = 2^d - 1$. $\square$

Lemma 3.9 (Forced-subtree inertness). *Suppose a terminal node denoting signed int32 value v is replaced by a complete forced continuation whose remaining internal nodes use feature zero and threshold $2^{31} - 1$, and whose descendant leaves all contain that same v . For every $q \in \mathbb{I}_{32}^F$, traversal through that continuation returns v ; hence the replacement preserves the terminal value over the entire packed-input domain.*

Proof. Definitions 3.1 and 3.2 give $F \geq 1$, so feature zero exists. Because $q_0 \leq 2^{31} - 1$, the strict predicate $q_0 > 2^{31} - 1$ is false at every inserted node and traversal follows the left child at each remaining level. The reached descendant leaf contains v . Moreover, assigning v to every descendant leaf makes the value independent even of the particular descendant selected. Thus the forced continuation cannot change the tree output for any int32 input vector. $\square$

Theorem 3.10 (Finite work and logical traversal state). *For a canonical v1 model, inference performs exactly Td node decisions and T leaf reads. For a canonical v2 model, inference performs exactly KRd node decisions and KR leaf reads. An abstract streaming evaluator that does not buffer the model needs $O(1)$ scalar logical traversal state and $O(K)$ vector logical traversal state, excluding input storage and implementation-specific read buffers. Model space is $24 + Tb(d)$ for v1 and $24 + 4K + KRb(d)$ for v2.*

Proof. Each complete tree executes the loop in Equation 5 exactly d times, followed by one leaf read. The scalar outer loop visits T trees. The vector loops visit R trees for each of K outputs. No inference loop is recursive or conditioned on a model value other than the bounded header counts. A streaming evaluator stores one scalar accumulator or a length- K result vector together with a constant number of indices and decoded primitive values. The model-space formulae follow from Equation 2. $\square$

Remark 3.11 (Bounded does not imply affordable). The theorem establishes a finite operation count. It does not guarantee that an arbitrary encoded dimension fits a transaction gas limit. The publication profile and empirical gas envelope must be reported separately. The deployed Solidity boundary-reader allocates a fresh dynamic byte buffer when one primitive spans two chunks. Since EVM memory allocation is monotonic within a call, its transient memory may grow linearly with the number of such boundary reads even though every primitive read is at most 32 bytes. This is an implementation cost, not additional logical traversal state.

3.4 Cross-runtime equivalence

A *conforming forest evaluator* uses the specified little-endian signed/unsigned decoding and strict comparison, accumulates the mathematical integer sums below without overflow or loss of precision, and selects the lowest index among equal maximum logits when returning a class. A conforming local evaluator first validates the version-specific canonical definition. A *conforming EVM evaluator* at (σ, r) obtains every core primitive through $\text{Read}_{\sigma, M}$, uses the version-aware registry relations below, and otherwise applies the same forest semantics. These are specification definitions; source inspection and finite compiled/deployed tests separately assess the deployed `ForestRuntime` path. Client-side preprocessing may construct the packed integer vector, but real-number quantization is not part of the theorem. Applying the theorem to an external endpoint additionally requires its enabled, authorization, and fee conditions to reach the internal evaluator with the packed input unchanged.

For a scalar model, define the integer function

$$f_B(q) = a_0 + \sum_{t=0}^{T-1} v_{t, \pi_t(q)}, \quad (7)$$

where $\pi_t(q)$ is the leaf selected by Equation 5. For a vector model,

$$f_{B,k}(q) = a_{0,k} + \sum_{t=0}^{R-1} v_{k,t, \pi_{k,t}(q)}. \quad (8)$$

Theorem 3.12 (Specification-level conforming-evaluator equivalence). *Let B be canonical under Definition 3.1 or Definition 3.2, let the EVM manifest satisfy Definition 3.3 at the same (σ, r) used for evaluation, and require the following nine version-aware relations for r*

$$\begin{aligned} nFeatures &= F, & nTrees &= \begin{cases} T & (v1), \\ KR & (v2), \end{cases} & depth &= d, \\ baseQ &= \begin{cases} a_0 & (v1), \\ 0 & (v2 \text{ registry convention}), \end{cases} & scaleQ &= s, & tablePtr &= P, \\ totalBytes &= \ell, & numChunks &= N, & chunkSize &= c. \end{aligned}$$

Let every evaluator receive the identical $4F$ -byte sequence formed by packing $q \in \mathbb{I}_{32}^F$ as signed little-endian int32 values. Then conforming local and EVM evaluators return $f_B(q)$ for v1 and $(f_{B,k}(q))_{k=0}^{K-1}$ for v2. Their v2 class operation returns

$$\left(\min_{0 \leq k < K} \arg \max f_{B,k}(q), \max_{0 \leq k < K} f_{B,k}(q) \right),$$

where the minimum implements the specified lowest-index tie rule.

Proof. Lemma 3.5 gives identical serialized bytes. The v2 EVM and local evaluators decode the same header, node, threshold, and leaf integers. The conforming v1 EVM evaluator uses registered inference fields rather than parsing the header; the theorem's nine-relation premise makes those fields identical to the local decoder's values, while the node, threshold, and leaf integers still follow from Lemma 3.5. At the root, every evaluator compares the same q_f and θ_q using the same strict predicate. If their indices agree before a level, the identical predicate and Equation 5 give the same next index. Induction and Corollary 3.8 give the same selected leaf for every tree. Finally, Equations 7 and 8 add the same finite integer sequence to the same base term, with the exact-accumulation premise preserving that mathematical sum. Applying the same lowest-index argmax scan to the identical v2 vector gives the same class index and maximum integer logit. $\square$

Remark 3.13. This handwritten theorem concerns conforming evaluator specifications in one state and record. Source inspection supports the mapping from the deployed Solidity path to that specification; compiled tests and pinned observations supply finite implementation evidence. None is a mechanized EVM-bytecode proof, verified Solidity-to-bytecode refinement, or source-to-live-bytecode provenance proof.

Proposition 3.14 (Exact JavaScript accumulation under the deployment profile). *All integer accumulators permitted by the canonical `ModelStore/registry` profile are strictly within JavaScript's exact integer range.*

Proof. For v1, the registry's 16-bit tree count gives

$$|f_B(q)| \leq (65535 + 1)2^{31} = 2^{47} < 2^{53}.$$

For v2 with $d \geq 1$, Equation 2 gives $b(d) \geq 16$. Lemma 3.4 bounds total core payload by $767 \cdot 24572 = 18846724$ bytes. Since $K \geq 2$,

$$R < \frac{18846724}{2 \cdot 16} < 589000.$$

Hence every output and every partial sum obeys

$$|f_{B,k}(q)| < (589001)2^{31} < 2^{51} < 2^{53}.$$

All int32 inputs and summands are themselves exactly representable. $\square$

Remark 3.15. The theorem concerns inference from fixed bytes. Byte-identical training is a separate empirical claim because split scoring, gradients, and early stopping use floating-point arithmetic. It requires fixed preprocessing, operation order, feature/class order, seeded random-number consumption, and compiler flags, plus a parity test matrix.

3.5 Fixed-point approximation

For finite $z \in \mathbb{R}$ and integer $s > 0$, define the abstract exact-real quantizer with JavaScript's half-toward-positive-infinity tie rule

$$Q_s(z) = \text{clamp}_{\mathbb{I}_{32}} \left(\left\lfloor sz + \frac{1}{2} \right\rfloor \right). \quad (9)$$

This definition includes the asymmetric negative-half convention of `Math.round`; for example, $Q_1(-1.5) = -1$.

This is a mathematical abstraction. Production JavaScript first computes the binary64 product $\text{fl}(sz) = sz + \delta_s(z)$ and then applies `Math.round`. The two integers agree when that multiplication perturbation does not cross an integer rounding boundary. Without that premise, and provided the rounded production result remains in the int32 range, the direct implementation bound is

$$\left| \frac{\text{Math.round}(\text{fl}(sz))}{s} - z \right| \leq \frac{1/2 + |\delta_s(z)|}{s},$$

which can be slightly larger than $1/(2s)$. If int32 saturation occurs, neither bound applies.

Lemma 3.16 (Abstract single-value quantization error). *If Equation 9 does not saturate, then*

$$\left| \frac{Q_s(z)}{s} - z \right| \leq \frac{1}{2s}.$$

Proof. For $u = sz$, the integer $\lfloor u + 1/2 \rfloor$ lies in $[u - 1/2, u + 1/2]$. Divide by positive s . □

Theorem 3.17 (Stable-path output error). *Consider an ideal scalar forest whose base and the T leaves selected for input x are independently quantized by Equation 9 without saturation at common scale s . If quantization does not change any selected path, the dequantized GL1F output $\hat{y}$ satisfies*

$$|\hat{y} - y| \leq \frac{T + 1}{2s}.$$

For one output of a v2 model, replace T by R .

Proof. Apply Lemma 3.16 to the base and every selected leaf, then use the triangle inequality. □

Lemma 3.18 (Sufficient split-margin condition). *Suppose neither x nor threshold θ saturates. If*

$$|x - \theta| > \frac{1}{s},$$

then the truth value of $x > \theta$ equals that of $Q_s(x) > Q_s(\theta)$.

Proof. If $x > \theta + 1/s$, Lemma 3.16 implies $Q_s(x)/s \geq x - 1/(2s) > \theta + 1/(2s) \geq Q_s(\theta)/s$. The reverse ordering is analogous. □

Corollary 3.19 (Argmax stability). *Assume stable paths and unsaturated base/leaf quantization for a v2 model. If the difference between the largest and second-largest ideal logits exceeds*

$$\frac{R + 1}{s},$$

the GL1F integer argmax returns the same class. Equal integer logits are resolved deterministically in favor of the lowest class index.

Proof. Theorem 3.17 perturbs each of the two logits by at most $(R + 1)/(2s)$. Their order therefore cannot reverse under the stated margin. The implementation replaces the incumbent only on strict improvement, which establishes the tie rule. □

Remark 3.20 (Why a global error bound needs a margin). Near a split, quantization can select a different leaf. The difference between two leaves is not bounded by the rounding error alone. Therefore Theorem 3.17 must not be quoted without its stable-path condition unless an additional bound on alternate-leaf values is supplied. For production binary64 preprocessing, replace every $1/(2s)$ term above by $(1/2 + |\delta_s(z)|)/s$ for the corresponding quantized value, unless the implementation is shown to emit the same integer as the abstract quantizer; either alternative still requires that the corresponding int32 clamp not saturate. Provided neither operand saturates, a sufficient production split margin is therefore

$$|x - \theta| > \frac{1 + |\delta_s(x)| + |\delta_s(\theta)|}{s}.$$

3.6 Limits of what the current registry proves

For clarity, the live verifier names nine relations rather than using an undefined “applicable metadata” category. It checks

$$\begin{aligned} nFeatures &= F, & nTrees &= \begin{cases} T & (v1), \\ KR & (v2), \end{cases} & depth &= d, \\ baseQ &= \begin{cases} a_0 & (v1), \\ 0 & (v2 \text{ reserved-field convention}), \end{cases} & scaleQ &= s, \\ tablePtr &= P, & totalBytes &= \ell, & numChunks &= N, & chunkSize &= c. \end{aligned}$$

The first five reconcile runtime registry metadata with the decoded core; the last four reconcile the runtime and byte-information registry getters with the storage manifest.

Proposition 3.21 (No unconditional content binding). *For the deployed registry entry point, the existence of a record under identifier h does not imply $h = \text{keccak256}(B)$ for the bytes reconstructed from its pointer table.*

Proof. The registration predicate checks only that $h \neq 0$ and that no prior record exists under h ; it accepts h and the pointer metadata as independent arguments. Choose any nonzero h and any accepted pointer tuple reconstructing B with $h \neq \text{keccak256}(B)$. The call can satisfy the remaining fee, metadata, ToS, and licence checks. Thus a valid transaction is a counterexample to the implication. $\square$

Remark 3.22 (Executable assurance-boundary witness). The isolated-chain integration test makes the counterexample concrete. The unmodified registry accepted this identifier:

```
0x60c713e19362bcb16b04fb6fc611821a
8a3fb90e1698490bf4819c2c44838875
```

for a pointer tuple whose 104 reconstructed bytes have hash

```
0xeb0ee33a46399a8592b6676035a7e580
c1f90e7761a2eb1f4a56d0f9f0e4ab18.
```

The runtime followed that record and returned integer prediction 1043. The production JavaScript chain loader now recomputes $\text{keccak256}(B)$ after exact reconstruction and rejects this mismatch before returning model bytes. This client-side check mitigates the deployed assurance gap; it does not change the registry predicate.

Corollary 3.23 (Need for independent verification). *Theorem 3.12 applies to a registry entry only after an independent verifier establishes Definitions 3.1 or 3.2, Definition 3.3, and all nine relations above. If the registry key is additionally claimed as a content identifier, that separate deployment-identity claim requires $h = \text{keccak256}(B)$; this equality is not a premise of execution equivalence for the reconstructed bytes.*

Proposition 3.24 (Open model storage permits independent local inference). *Under the documented GL1C storage design, an observer with chain-read access can reconstruct the intentionally public model and evaluate it locally, independently of the canonical runtime endpoint.*

Proof. The pointer table and every chunk are public EVM runtime code retrievable by address. Definition 3.3 gives a constructive algorithm for reassembling B . Equations 7 and 8 then define a local evaluator using only public integers. No secret required by evaluation is held by the fee-gated contract. $\square$

Remark 3.25. Mode-2 payment can still meter use of an official transaction endpoint, produce an on-chain event, or support service-level economics. It is not model encryption, copy protection, or exclusive inference.

3.7 Proof obligations for a publication-grade deployment

The strongest reproducibility record should contain:

1. the exact core bytes and both SHA-256 and Keccak-256 digests;
2. all chunk addresses, runtime code hashes, ordered table address, c , N , and ℓ ;
3. a strict parser report covering magic, version, reserved fields, dimensions, exact length, and every feature index;
4. the nine named registry/core/storage relations above, plus any separately claimed NFT feature metadata;
5. packed feature vectors and integer outputs for conformance cases;
6. source revision, compiler version and binary digest, optimizer settings, IR setting, EVM target, creation bytecode, and deployed bytecode hash; and
7. a parity matrix that distinguishes byte-identical training evidence from the inference theorem proved above.

These obligations make the claim falsifiable: an independent party can either reproduce every integer output or identify the first violated premise.

4 Executable evidence

4.1 Three-engine training parity

The publication suite invokes the production browser worker under Node, compiles the production C++ trainer, invokes the Python CLI, and compares the core GL1F bytes. It covers regression, binary, multiclass, and multilabel objectives; linear and quantile binning; weighting; stratification; early stopping; fixed-budget refitting; learning-rate schedules; seed zero; repeatability; v1/v2 inference; rounding boundaries; packaging; and hostile stream rejection.

The suite identified a real operation-order discrepancy. The browser computed linear histogram bins by multiplying by a cached reciprocal, while Python and C++ divided by the range before multiplication. The algebra is real-number equivalent but not IEEE-754 equivalent at every bin boundary. Replacing the reciprocal multiplication with the same divide-then-multiply order restored all tested three-engine byte comparisons.

An additional generated boundary case then exposed a second finite witness. For a generated 4,000-row, 12-feature, four-class profile, Python accumulated the softmax denominator from binary32-rounded exponentials while JavaScript and C++ accumulated binary64 exponentials. One byte in the 22,120-byte Python core differed, corresponding to a one-Q-unit leaf change. The corrected profile uses binary64 exponentials, an ascending-class-index binary64 denominator sum, a binary32 numerator, binary64 division, and a binary32 result. All three engines now emit 58e3d9e63c5dd1313872e0c5a07c229b071272f1137f393a0384601329a432eb for that witness. This is a maintained regression result, not a universal claim about every platform or mathematical library.

4.2 Reference performance observation

Table 2: Recorded end-to-end trainer median (IQR width) in seconds; 30 timed repetitions per engine and task after one warm-up.

Workload	Python median	C++ median	JavaScript/Node median
Regression	0.2847 (0.0183)	0.0348 (0.0027)	0.1355 (0.0105)
Binary classification	0.3074 (0.0146)	0.0421 (0.0028)	0.1503 (0.0082)

Each run used 3,000 rows, 12 features, 60 depth-4 trees, and 30 measured repetitions per engine and task after one warm-up. Startup, parsing, training, serialization, and output writing are included. All three engines produced identical 11,064-byte cores. Ratios of the observed Python and C++ medians were 8.19 and 7.30; they are workload-specific measurements, not universal speed claims.

4.3 Isolated EVM integration

The local integration test compiles the unmodified contracts with the pinned profile, deploys ModelStore, ModelRegistry, ModelNFT, and ForestRuntime to an isolated Ganache chain, stores minimal canonical v1 and v2 models through 17-byte GL1C chunks and pointer tables, reconstructs the code, verifies both model commitments, and compares independent bigint reference inference with scalar, vector, and class EVM results on six equality/extreme boundary vectors.

Table 3: Isolated local-EVM correctness-fixture measurements.

Measurement	v1 scalar	v2 vector/class
Core bytes / chunks	104 / 7	156 / 10
Mean view gas estimate	77,600	109,011 vector; 108,412 class
Transaction inference gas	79,573	115,527

Equal vector logits deterministically select class zero, matching the specified lowest-index tie rule.

4.4 Local-EVM scalar scaling

A complementary benchmark publishes six deterministic two-feature v1 shapes through 20,000-byte chunks and applies the same 12 seeded vectors to each shape. All 72 reference/runtime profile-vector comparisons matched.

For the tested scalar decoder, tree-body model-code reads—feature and threshold reads at each internal node and one leaf read per tree—equal $R_{\text{tree}} = T(2d + 1)$. This excludes registry, core-header, and pointer-resolution accesses. A descriptive ordinary least-squares summary of the six unrounded means was approximately

$$\widehat{\text{gas}} \approx 42,900 + 1,708.3R_{\text{tree}},$$

Table 4: Local scalar scaling observations. Means are rounded to the nearest gas unit; raw observations and unrounded statistics are [archived](#).

Trees	Depth	Core bytes	Tree-body reads	Mean gas
20	3	1,784	140	286,247
50	3	4,424	350	644,454
50	4	9,224	450	810,777
100	4	18,424	900	1,576,945
200	4	36,824	1,800	3,121,210
50	6	38,024	650	1,146,414

with $R^2 \approx 0.99998$ and maximum absolute relative residual 1.45%. This is a six-point description under one recorded Solidity 0.8.20/Ganache 7.9.2 profile, not a gas theorem, mined-transaction result, or live-network fee forecast.

5 Pinned deployment witness

At GenesisL1 block 13,342,043, recorded hash 0xffd825db1bb2534052a604db9584d361111d8bc9e19d753b0ee3861bf320d1b9, the strict witness reconstructed all 31,185,324 bytes of 12 active models. Every exact pointer-table/chunk shape passed, every recomputed `keccak256(core)` matched its model identifier, every strict header passed each applicable registry/header and storage check, and every local zero-vector integer output matched the deployed EVM read-call output. The block hash was re-read after all witness calls and remained unchanged.

#	Model	Ver.	F	d	Trees	Bytes / chunks
1	Iris model	2	4	3	1,800	158,436 / 7
2	Raisin classification	1	7	5	169	63,568 / 3
3	Boston housing prices	1	13	6	196	148,984 / 7
4	PV efficiency, 57° North	1	4	3	275	24,224 / 2
5	Bitcoin 24 h volatility	1	53	5	20	7,544 / 1
6	ETH +2% in 5 h	1	26	8	1,496	4,583,768 / 191
7	ETH −2% in 5 h	1	27	8	2,500	7,660,024 / 320
8	XRP +2% in 5 h	1	33	8	1,008	3,088,536 / 129
9	XRP −2% in 5 h	1	33	9	1,552	9,523,096 / 397
10	ZEC +4% in 5 h	1	33	7	1,789	2,733,616 / 114
11	ZEC −4% in 5 h	1	33	8	1,042	3,192,712 / 134
12	ZEC ±4% volatility gate	1	47	3	9	816 / 1

The recorded witness summary is byte/execution evidence, not predictive validation. Its raw provider responses are not included, so the live-state values are not independently reproducible from this repository alone. An `eth_call` is not a transaction receipt or consensus proof, and two creator addresses do not prove two independent people or organizations.

6 Open-protocol and registry boundary

GL1F is an open protocol whose model bytes are intentionally public. Paid mode may meter the canonical transaction/read endpoint, event trail, or integrated service; it is not model encryption and does not prevent independent local evaluation.

The deployed registry records a caller-supplied model identifier and manifest without recomputing the entire core on-chain. This does not invalidate the canonical publisher or the 12 live records—the strict witness confirms all 12. It means that arbitrary future entries require the same independent verifier unless a later registry enforces the binding. The consequence is a registration-integrity and availability boundary.

7 Scientific limits and falsifiability

The format, tests, proofs, NFT, and chain witness do not establish:

- predictive accuracy, calibration, external validity, or causal truth;
- absence of leakage, sampling bias, data drift, or subgroup harm;
- authorship, legal intellectual-property ownership, or licence enforceability;

- secrecy of model bytes, feature vectors, or outputs;
- correctness of an RPC response without the stated node/state assumptions; or
- suitability for clinical, financial, or other high-stakes use.

Application-level claims require dataset provenance, a prespecified evaluation protocol, baselines, ablations, uncertainty estimates, calibration, and domain-specific ethical and safety analysis. These materials are outside the scope of this systems report.

8 Reproduction commands

```
npm ci
python3 -m unittest -v tests.publication.test_publication
python3 -m unittest -v tests.contracts_publication.test_formal_properties
npm run test:contracts
npm run test:evm
npm run benchmark:evm
python3 tests/ui_static_check.py
python3 benchmarks/publication_benchmark.py \
  --out benchmarks/results/publication_benchmark.json
```

The public reproduction set contains the local evidence and the read-only live-witness method. It does not contain the raw provider responses required for an offline reproduction of the historical live-state summary.

See REPRODUCIBILITY.md for the exact environment, commands, checked-in records, and evidence boundary. The language-neutral binary grammar and publication profile appear in FORMAT_SPEC.md.

9 Conclusion

GL1F's scientific leverage comes from reducing the semantic gap between model artifact and execution. The tree grammar is fixed-width enough to count, the integer semantics are simple enough to prove, the three training surfaces are close enough to test at the byte level, and the published code is open enough to reconstruct independently. The resulting model is not an oracle of truth; it is a public computational instrument whose exact behavior can be inspected, challenged, and deterministically reevaluated under explicit premises.